Auction-Based Consensus for Crowd-Shipping

Zohar Monsonego, Yael Haim-Shuv
Shamoon College of Engineering, Ashdod, Israel

March 10, 2026

Abstract

This study presents BidDrop, a mobile and decentralized system that integrates crowd-shipping and ride-sharing through blockchain technology. The platform is built on Substrate and employs an Auction-based consensus mechanism for assignment selection, with payments and security managed by an on-chain smart contract. BidDrop delivers dynamic, fair matching between senders, passengers, and drivers, while ensuring full bid transparency, on-chain data integrity, and verifiable on-chain attestations. Rather than relying on a centralized matching engine, the system demonstrates how critical assignment and payment decisions can be executed directly within the blockchain runtime, while operational data remains off-chain for efficiency. This hybrid design highlights the practical benefits of embedding auction-based consensus into real logistics workflows, as well as the challenges involved in extending such decentralized coordination mechanisms to large-scale deployments.

Contents

1 Introduction **5**

2 Market Research **9**

3 Literature review **12**
3.1 *Crowd-Shipping* 12
3.2 *Blockchain* 12
3.2.1 Consensus Mechanisms 12
3.2.2 Smart Contracts 13
3.3 *Auction Theory* 13
3.4 *IDA* Algorithm* 14

4 System Definition **16**

5 System Implementation **17**

6 End-to-End Flow Demonstration **21**

7 Discussion and Conclusion **32**

8 Appendices **34**
8.1 Requirements Document (JIRA Link) 34
8.2 Source Code Repository (GitHub Link) 34
8.3 Risk Management 34
8.4 System Testing and Validation 35

9 References **68**

1 Introduction

The rapid growth of e-commerce and on-demand mobility has intensified the need for efficient and environmentally responsible delivery solutions. Traditional courier-based logistics models struggle with rising fuel and insurance costs, traffic congestion, and limited scalability, while community ride-sharing platforms reveal opportunities to leverage existing traffic flows for short-distance deliveries and passenger transport. This motivates the exploration of crowd-shipping models in which independent drivers use their planned routes to carry packages or passengers, improving efficiency without adding significant travel overhead. Despite these advantages, trust and security remain major obstacles. Unlike regulated couriers, independent drivers lack formal accountability, raising concerns about delivery reliability, package integrity, and potential misuse of the service. Ensuring safe and timely deliveries therefore requires strong verification, tamper-proof records, and transparent coordination mechanisms that users can rely on.

To address these challenges, this study investigates the integration of blockchain technology with an auction-based coordination mechanism for decentralized delivery and ride-sharing. Blockchain provides transparency, immutability, and verifiable transaction histories, while the auction model enables dynamic and efficient matching between drivers and requests. Together, these elements form a secure and decentralized foundation for crowd-shipping, aiming to enhance reliability, reduce operational overhead, and improve coordination in modern mobility and logistics networks.

Auction-Based Consensus for a Blockchain-Based Crowd-Shipping and Ride-Sharing Platform

This work presents BidDrop, a decentralized crowd-shipping and ride-sharing platform built on the Substrate [19] framework. Substrate was selected for its modular and extensible architecture, which allows developers to modify consensus and runtime components with relative ease—an essential requirement for integrating our custom auction-based PoBA (Proof-of-Bid-Assignment) consensus mechanism directly into the blockchain runtime. PoBA reflects an auction-like logic: nodes independently construct and submit bids representing their proposed assignments for the current delivery or ride-sharing requests. Each bid includes a score that reflects its quality, and the highest-scoring bid is selected as the winning assignment for that slot. This produces a transparent and verifiable proof of how each assignment was chosen.

BidDrop enables users to create delivery or ride requests, publish driver availability, receive decentralized assignment decisions, track delivery progress, confirm successful handovers, and securely manage payments—all through a unified mobile application. Drivers can accept matched assignments along their existing routes, while senders and passengers benefit from transparent coordination and verifiable on-chain settlement.

The platform integrates a mobile app, backend service, PostgreSQL database, and an on-chain runtime that coordinates assignments using PoBA. Each node independently computes matching proposals through an IDA*(Iterative Deepening A*)-based algorithm, and the Aura(Authority Round) [7]-selected block author finalizes the winning proposal for each slot by invoking the PoBA finalization logic. The blockchain records these finalized assignments immutably, ensuring fairness, resistance to tampering, and decentralized verification.

To support secure payment flows, BidDrop includes a native on-chain escrow pallet implemented directly within the Substrate runtime. This component provides smart-contract

functionality for creating, holding, and releasing payments, tightly integrated with the assignment and confirmation logic of the platform. Deliveries are thus matched and settled through a fully verifiable and tamper-proof on-chain process. Operational data such as requests, offers, and user metadata remains off-chain for efficiency, while only trust-critical states—finalized assignments and escrow records—are preserved on the blockchain.

A full illustration of the architecture appears in Figure [1], which outlines the platform's main components, the flow of information between them, and the modular separation between user interaction, backend logic, blockchain-level consensus, and payment operations. The figure highlights how mobile clients communicate with backend services, how operational data is maintained in the database, and how nodes participate in assignment selection and escrow processing through dedicated runtime pallets.

Figure [2] complements this overview by presenting the wallet-based authentication flow used throughout the system. It shows how users connect, obtain a signed challenge from the backend, and authenticate securely without relying on traditional passwords. This authentication layer serves as the entry point for all subsequent interactions within the BidDrop system.

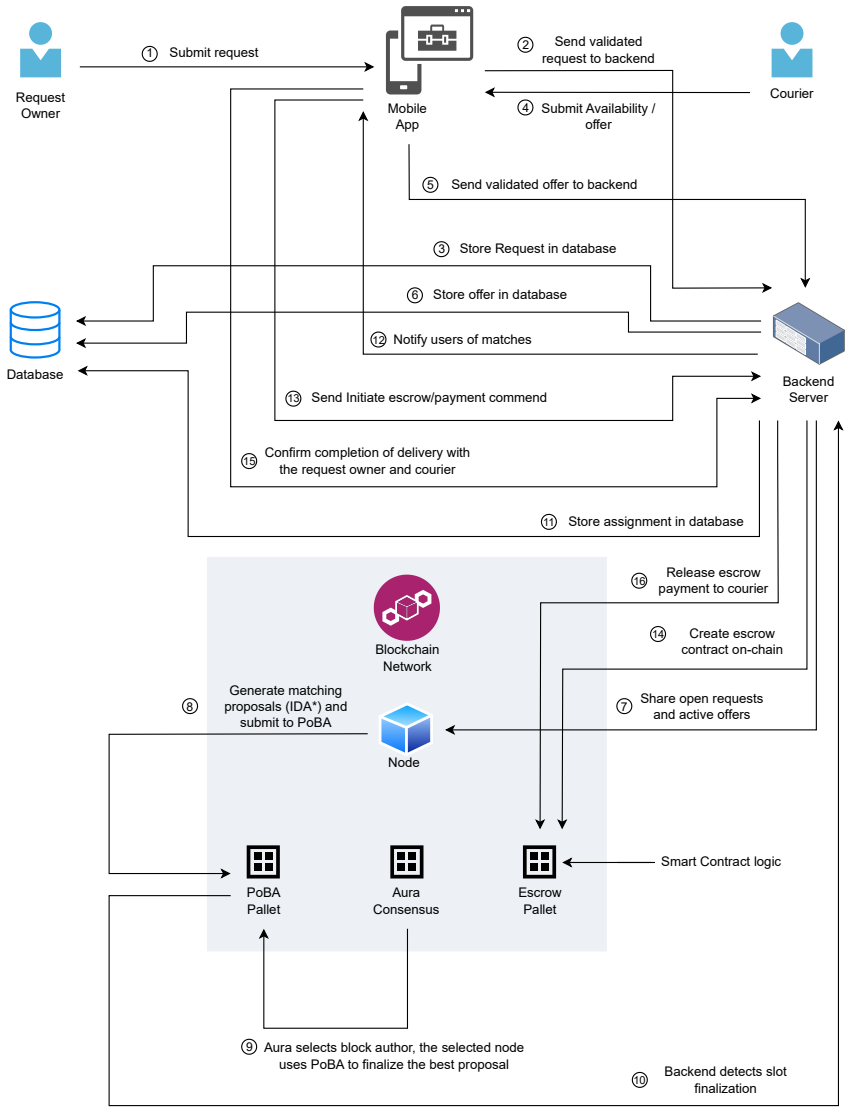

Figure 1: Overall System Flow of the BidDrop Platform

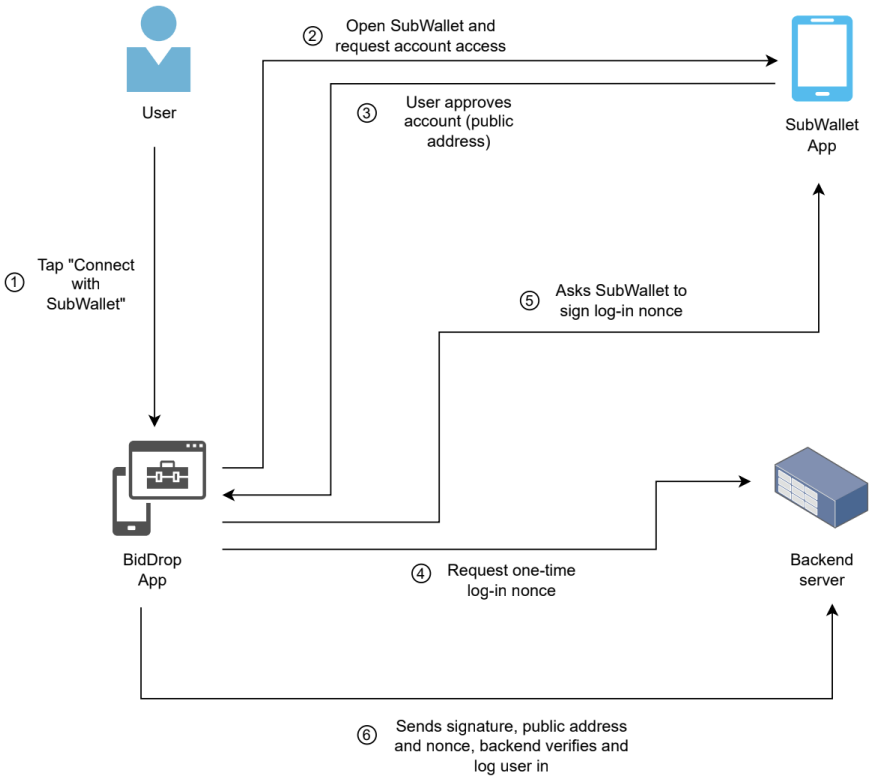

Figure 2: Wallet-Based Authentication Flow in the BidDrop Platform

2 Market Research

Our system is a smart, collaborative delivery platform based on blockchain technology, utilizing a consensus mechanism based on auctions. To conduct market research, we will focus on blockchain-based systems or collaborative models, and we will use metrics such as

- Use of blockchain technology
- Use of consensus mechanisms
- User incentives - token economy
- Autonomous delivery routing
- Sharing of private manpower
- Supply chain transparency
- Decentralization of data management
- Delivery service available

We will present an overview of systems currently available in the market that are similar to ours:

- **Morpheus.Network:**
  Morpheus.Network is a blockchain-based platform designed to optimize the global supply chain through smart contracts, digital documentation, and process automation. It does not provide delivery services itself, but serves as an umbrella system that connects various entities in the supply chain, such as manufacturers, suppliers, carriers, customs authorities, and customers, in a secure, transparent and automated manner. Morpheus.Network enables real-time tracking of shipments, integrates with systems like SAP and Oracle, uses the MRPH token as a payment method for internal processes on the platform, and implements smart contracts to automate processes (e.g., automatic payment release upon delivery arrival). It operates on a Proof of Authority (PoA) consensus—not a public network like Ethereum—but a secured network with defined authorities. Morpheus.Network is not a classic collaborative delivery system (P2P); it lacks private couriers receiving payment for deliveries and does not have an open marketplace for deliveries between users.

- **Fetch.ai:**

  Fetch.ai is a blockchain-based platform that combines artificial intelligence (AI) with smart contracts, enabling autonomous entities (i.e., "smart agents") to operate, communicate, and conduct transactions without direct human intervention. Fetch.ai has a model similar to the concept of collaborative deliveries:

  - **Autonomous Economic Agents (AEAs):** Each participant in the system—customer, courier, restaurant, or store—represents a "smart agent" acting on their behalf. The agent seeks optimal matches for deliveries, such as a nearby courier with high availability.
  - **Consensus and Smart Contracts:** Everything operates on the blockchain; once an agent finds a match for a delivery, a smart contract is created that stipulates the delivery terms (price, time, destination, etc.).

- **Decentralized and Dynamic Market:** There is no intermediary like a courier company—the users operate in a P2P manner, and incentives are provided through the FET token.
- **Decentralized Search Engine (Open Economic Framework):** Allows for quick searches for matches—similar to Uber but without a central server—conducted by agents who publish their needs on the network.

- **DHL Blockchain Pilot:**
  This is an experimental initiative by the international courier company DHL in collaboration with Accenture, examining how blockchain technology can enhance the global supply chain. The goal was to create a transparent, private blockchain-based, secure, and automated system for managing international deliveries— including tracking packages, approvals, accountability chains, and more. In 2018, they developed a blockchain-based prototype for tracking the supply chain of pharmaceuticals. The aim was to improve transparency and traceability in the shipping process, especially in the pharmaceutical sector, to prevent counterfeiting and ensure product reliability. The system includes blockchain-based tracking—the system records every stage of the shipping process, from manufacturer to consumer, in an immutable distributed ledger, enabling information sharing among various stakeholders—manufacturers, warehouses, distributors, pharmacies, hospitals, and doctors—providing access to reliable and consistent information on the shipment status, and addressing counterfeiting—through precise tracking, it is possible to identify and prevent counterfeit drugs from entering the supply chain. During simulations, it was found that the system could handle over 7 billion unique serial numbers and execute 1,500 transactions per second, indicating high scalability.

- **UberEats:**
  UberEats is a delivery platform operated by Uber that allows customers to order food from local restaurants via an app or website. The system operates on a centralized model, where Uber manages the entire process—from ordering food to delivery. Customers can choose from a wide variety of restaurants, place orders, and track deliveries in real-time. UberEats charges fees from restaurants and delivery fees from customers. The system focuses on fast and convenient service but does not offer the transparency found in decentralized systems and does not utilize blockchain technology.

- **WOLT:**
  WOLT is a delivery platform offering services similar to those of UberEats, with an emphasis on food and restaurants. WOLT operates in many countries and provides a user-friendly ordering experience. Customers can order food from a wide variety of restaurants, and WOLT offers fast deliveries and quality customer service. The system emphasizes collaboration with local restaurants and provides discounts and special offers for repeat customers. Like UberEats, WOLT does not use blockchain technology and does not operate on a decentralized model.

- **ASCOMY:**
  ASCOMY is an advanced Israeli system for managing and planning daily distribution routes, providing full control over the distribution network in real-time through a SUPERVISION DASHBOARD. This system is designed specifically for distribution companies to optimize their shipping processes, rather than functioning as a shipping

service itself. The system offers maximum optimization for planning ideal routes, is flexible for route updates, and presents various simulations. It includes a scheduling module that performs automatic optimization of distribution routes, considering truck data, drivers, dates, supply units, time windows, anchor points, road data, and road condition limitations. The feedback tools provide tracking of task progress in the field and offer real-time solutions. The system is operated by drivers through a user-friendly app installed on Android-based smartphones, managing the entire process, from picking up goods from the warehouse to the truck's return after delivery completion. The app includes a planned route, connection to WAZE, digital signature, delivery reporting, return collection, and billing. ASCOMY employs blockchain technology to ensure data transparency and security. ASCOMY operates based on a consensus mechanism driven by auctions, aimed at selecting the most suitable players for each delivery while reducing costs and improving efficiency. It offers user incentives through a unique token that allows them to earn for each delivery they complete or receive. It enables private partners (like couriers or drivers) to join the system and perform deliveries, thereby increasing the available human resources within the system. It also employs advanced technologies for autonomous delivery routing, using artificial intelligence and smart models for data analysis. ASCOMY provides transparency in the supply chain—all transactions and processes are recorded on the blockchain, ensuring full transparency for all involved parties.

Feature	**Morpheus Network**	**Fetch.ai DHL**	**Blockchain Pilot**	**UberEats**	**Wolt**	**ASCOMY**	**Our System**
Use blockchain technology	V	V	V	X	X	V	V
Use of consensus mechanisms	V	V	V	X	X	V	V
User incentives	V	V	X	X	V	V	V
Sharing of private drivers	X	V	X	V	V	V	V
Autonomous smart routing	V	V	X	X	X	V	V
Full transparency in the network	V	V	V	X	X	V	V
Decentralization of data	V	V	V	X	X	V	V
Delivery service available	X	X	V	V	V	X	V

Table 1: Comparison of Delivery and Blockchain Systems

3 Literature review

This literature review begins by outlining the fundamental concepts that underpin decentralized logistics systems. Key terms such as crowd-shipping, blockchain, consensus protocols, and smart contracts are defined and discussed, providing essential background for understanding the technological landscape. Presenting these concepts at the outset enables a comprehensive exploration of their roles, challenges, and integration within the context of innovative delivery models.

3.1 *Crowd-Shipping*

Crowd-shipping is gaining attention as a key solution in last-mile delivery. Last-mile crowd-shipping is a shipment delivery method involving occasional drivers, or crowd-shippers, who act as couriers during their daily activities. It includes two key stakeholders: crowd-shippers and shipment owners—all connected through an online platform, bringing benefits to each stakeholder as well as to society and the environment [2].

Despite its benefits, crowd-shipping systems face several persistent challenges. Pricing is often inefficient, as senders may receive an overwhelming number of bids, creating decision fatigue and motivating the use of structured auction mechanisms. Another key challenge is task allocation: current CS platforms frequently rely on unsupervised matching processes that fail to consider route compatibility, courier availability, and delivery time windows [2]. Empirical studies, including analyses of platforms such as Uber and other on-demand delivery services, show that routing and matching are deeply interconnected, and that feasible assignments depend simultaneously on route structure, capacity constraints, and temporal requirements—factors that most existing systems do not integrate holistically [3].

3.2 *Blockchain*

Blockchain is a type of Distributed Ledger Technology (DLT) that ensures secure, transparent, and immutable storage of data. Each block in the blockchain contains a timestamp and a cryptographic link to the prior block, making it challenging to modify the data once it has been preserved. This structure creates a decentralized database maintained by a network of nodes that collectively authenticate and record transactions without relying on a central authority [8]. In crowd-shipping systems, this decentralized architecture is particularly valuable for ensuring transparency in auction processes and trust among participants. By recording bids and transactions immutably, senders, passengers, and couriers can rely on blockchain to prevent disputes or fraud.

3.2.1 Consensus Mechanisms

Consensus refers to the process of achieving agreement among a group of individuals or entities regarding a specific decision or action. Within blockchain systems, consensus ensures that all nodes in the network agree on the current state of the ledger and the validity of transactions [8].

Blockchain systems employ several consensus mechanisms, including Proof of Stake (PoS), Delegated Proof of Stake (DPoS), and Proof of Authority (PoA), each balancing scalability, decentralization, and security differently [1]. PoS and PoA aim to improve efficiency and energy consumption, while DPoS achieves high throughput by relying on elected

delegates—but at the cost of increased vulnerability to collusion due to their fixed identities [5].

Aura (Authority Round), a widely adopted PoA (Proof-Of-Authority) protocol, assigns block authorship in a fixed round-robin schedule among a predefined set of authorities. Time is divided into equal-length steps, and in each step a single in-turn author is selected deterministically. This predictable rotation minimizes coordination overhead and enables rapid block production, as each authority knows exactly when it is expected to produce a block. Aura is valued for its high efficiency, low computational cost, and simplicity [7].

Substrate

Substrate is a modular blockchain development framework that enables the creation of customized blockchains. Its architecture is designed for efficiency and flexibility, separating on-chain and off-chain responsibilities to reduce computational and storage overhead. Key features such as Off-Chain Workers and Off-Chain Storage allow intensive operations to be performed outside the blockchain state, significantly improving performance. Substrate is built around a modular design based on reusable and extensible pallets, which define the runtime logic of the chain; this architecture allows developers to add, modify, or replace functionality with minimal friction [6]. Substrate also integrates the Aura consensus protocol in many of its deployments, making it one of the widely used platforms that rely on this round-based Proof-of-Authority mechanism [7].

3.2.2 Smart Contracts

Smart Contracts are computer code embedded in a blockchain network that enables the automatic execution of agreements without human intervention. They operate as a pre-programmed "if-then" protocol that ensures that all contract conditions are met before an action (such as a payment or shipment release) is carried out. The technology relies on immutable transparency and cryptographic security, ensuring that all parties are bound by predefined terms without the possibility of tampering or breach. In the context of shipment management, smart contracts offer significant improvements by eliminating the need for intermediaries such as banks, logistics companies, or arbitration bodies, replacing them with autonomous code running on decentralized networks [9].

3.3 *Auction Theory*

Auctions serve as mechanisms for distributing goods or services through competitive bidding processes. Auctions are particularly valuable for gathering information from participants, offering insights into their individual valuations, preferences, or specific asymmetric knowledge they may possess. Each bidder typically holds a private valuation of the item being auctioned, shaped by their preferences, expertise, or external factors [4].

Auction-Based Consensus

A consensus-based auction approach is employed for decentralized task selection, combining the computational efficiency and robustness to inconsistencies in situational awareness offered by auction algorithms with the resilience to dynamic network topologies provided by consensus algorithms. Unlike previous consensus-based methods that focus on achieving agreement on the entire situational awareness, this approach uses a consensus routine

to establish agreement specifically on the winning bid values. This hybrid approach ensures robust task allocation, quick convergence, and adaptability in dynamic and uncertain environments [10].

3.4 *IDA* Algorithm*

The IDA* algorithm (Iterative Deepening A*) is an efficient extension of the classic A* algorithm, developed to overcome the memory limitations that characterize heuristic search in large state spaces. While A* maintains extensive data structures during the search and therefore requires significant storage space, IDA* combines the principles of A* with Iterative Deepening Depth-First Search (DFID), performing a series of searches with gradually increasing upper bounds [18].

Auction-Based consensus for a Blockchain based crowd-shipping systems

Several studies have investigated the operational and economic dynamics of crowd-shipping platforms. One line of research examined auction-based pricing and courier bidding behavior in real-world logistics systems [11], offering insights into how spatial strategies and price sensitivity influence the success of delivery bids. However, these analyses did not address trust, security, or decentralized verification—factors that are essential in open, crowd-based delivery environments.

Additional work proposed large-scale delivery matching models using combinatorial auctions and Vickrey-Clarke-Groves (VCG) mechanisms [12]. While these approaches improved task-allocation efficiency, they did not consider the technological challenges of establishing transparency, fraud resistance, or reliable validation when non-professional couriers are involved.

Simulation-based studies have further explored the matching between passengers and freight in crowd-shipping services [13], focusing primarily on logistical performance and algorithmic efficiency. Yet, these models paid limited attention to system integrity or the security mechanisms required to maintain user trust.

Other research employed agent-based modeling to analyze behavioral factors affecting last-mile crowd-shipping operations [14]. Although these studies enriched the understanding of operational dynamics, they did not incorporate auction mechanisms or blockchain-backed verification, leaving open questions regarding secure pricing processes and transaction guarantees.

Finally, investigations into bidding strategies and delivery performance highlighted the role of economic incentives in shaping courier participation [15]. Nonetheless, these works did not explore how decentralized technologies or trust mechanisms could mitigate fraud, enhance transparency, or ensure reliable coordination among participants.

Taken together, these studies highlight substantial progress in modeling, optimizing, and analyzing crowd-shipping operations, yet they also reveal persistent gaps in trust, security, and transparent verification. These limitations underscore the need for new approaches that integrate reliable coordination mechanisms with secure, decentralized technology—an avenue that the present research explores.

4 System Definition

The BidDrop platform is a blockchain-based crowd-shipping system, specifically designed to facilitate decentralized parcel delivery through the integration of dynamic ride-sharing opportunities and an auction matching mechanism. The platform comprises a mobile application, a backend service, and a dedicated Substrate-based blockchain network.

In this work, we introduce a custom consensus called Proof-of-Bid-Assignment (PoBA). PoBA is an auction-based mechanism where nodes compute optimized assignment proposals, and the Aura-selected block author finalizes the highest-scoring proposal for each slot. This embeds the matching algorithm directly into consensus.

BidDrop addresses two principal challenges in existing shipping platforms:

- It overcomes inefficiencies in matching delivery requests to available couriers; conventional systems often rely on manual or non-optimized allocation processes, which can result in suboptimal pairings and missed opportunities.
- The platform resolves issues of trust and transparency, providing users with an immutable, publicly verifiable record of all allocations and transactions via blockchain technology.

The system supports three main user roles:

- **Request Owners** — submit shipment or ride requests with specified locations, time windows, and maximum prices.
- **Couriers** — present availability and bid price ranges, offering service proposals that can be matched to requests.
- **Blockchain Nodes** — execute the IDA* algorithm, participate in the PoBA consensus process, determine optimal matches during each auction round.

BidDrop's core innovations include:

- A full-featured auction mechanism run directly on the blockchain, leveraging mathematical optimization (IDA*) for bid scoring across distance, price, and timing parameters.
- A consensus protocol (PoBA) that embeds assignment optimization directly into block production, enabling the blockchain to finalize the best available match transparently and deterministically.
- Complete on-chain transparency and immutability: all assignments, transaction records, and escrow contracts are permanently recorded on the blockchain.
- On-chain escrow: payments are programmatically locked and released only upon fulfillment of delivery conditions, enhancing trust and accountability.

Primary system components include:

- A mobile application (React Native/Expo) for user registration, request and bid submission, parcel tracking, and feedback management;

- Backend services (FastAPI + PostgreSQL) for user and request management, status tracking, blockchain synchronization, and database management of requests, bids, and assignments.
- A Substrate blockchain network featuring two custom pallets:
  - **PoBA Pallet**: defines the scoring and validation rules used when the Aura-selected block author finalizes the winning proposal for each auction round.
  - **Escrow Pallet**: implements the platform's smart contract logic for payment freezing and release, handling EscrowCreated and EscrowReleased events.

System boundaries:

BidDrop supports optimization-based automatic matching, fair selection through a decentralized auction mechanism, and secure payment handling via an on-chain escrow process. The system boundaries and internal interactions are illustrated in Figure [2] and Figure [1], which together describe the complete operational flow from user authentication to final payment release. Figure [2] presents the wallet-based login process: the user initiates "Connect with SubWallet," the wallet requests access to the public address, the user approves, the backend issues a one-time nonce, the wallet signs it, and the application returns the signature for cryptographic verification. This process defines the authentication boundary of the platform, which relies entirely on digital wallet signatures rather than passwords or centralized identity management.

Figure [1] details the full lifecycle of a delivery request. A request owner submits a delivery request, which is validated and stored in the database; similarly, couriers submit availability offers that undergo the same validation and storage. The backend then shares all open requests and active offers with the blockchain nodes, each of which independently computes a matching proposal using an IDA*-based optimization algorithm and submits the result to the PoBA mechanism. Aura selects the block author for the current slot, and the selected node finalizes the slot by choosing the highest-scoring proposal. After finalization, the backend detects the finalized slot, retrieves the selected assignment, stores it in the database, and notifies both parties of the match. The backend then triggers the initiation of an escrow, leading to the creation of an on-chain escrow contract. Once both the request owner and the courier confirm that the delivery has been completed, the escrow mechanism releases the payment to the courier, ensuring a transparent and tamper-proof settlement.

At the current stage, the platform does not include integration with external payment gateways (e.g., Visa, PayPal), predictive analytics for courier availability, real-time routing APIs, rating-based scoring adjustments within the auction, or a comprehensive dispute resolution interface. These limitations define the present operational boundaries of BidDrop and outline directions for future development.

5 System Implementation

Blockchain Network

BidDrop was developed atop the Substrate blockchain [19] framework. Substrate was chosen due to its fully extensible runtime and native support for custom consensus mechanisms. This flexibility enabled the implementation of the dual-consensus architecture required for

our research: Aura for block author selection and PoBA for optimal assignment finalization. In our deployment, BidDrop is instantiated as a permissioned network, where only pre-approved authority nodes participate in block authoring and validation, and all validators are explicitly configured rather than open to public join. This follows the characteristics of a private testnet: the validator set is fixed in advance, the runtime (including PoBA, Escrow, and related pallets) is executed solely by designated nodes, and the network operates in a controlled environment rather than as a public, permissionless chain. Unlike traditional public blockchains with rigid and open consensus, Substrate allows this controlled validator set and the associated runtime logic—such as custom block contents and on-chain algorithms—to be natively integrated.

Database

For persistent data management, the system utilizes PostgreSQL. This relational database was selected for its reliability and support for complex queries, all critical for handling user records, requests, offers, assignments, and escrow data. The schema includes dedicated tables for users, requests (indicating shipment or ride details), driver offers, match assignments confirmed via PoBA, and escrow transactions. Storing operational data primarily in the database rather than on-chain is intentional and essential. Request and offer information changes frequently, may include sensitive personal details, and must support filtering, sorting, indexing, and fast updates for real-time user experience. Writing every such update directly to the blockchain would be slow, expensive, and would expose private information publicly.

Backend Implementation

The backend is implemented in Python using the FastAPI framework, chosen for its asynchronous, non-blocking architecture, clean design, and high maintainability. FastAPI enables rapid development, modular organization of logic, and seamless interaction with both the PostgreSQL database and the Substrate blockchain node.

In this architecture, the backend acts as the application layer that manages all operational responsibilities that do not belong on-chain. It handles user authentication, request and offer creation, and updates to operational statuses.

After a slot is finalized on-chain, the backend listens to SlotFinalized and related events, retrieves the winning proposal, and applies the resulting assignments to its PostgreSQL database. It updates request and offer states and creates or modifies assignment records so that backend state stays aligned with the escrow pallet on-chain. This preserves a clear separation of concerns: the blockchain records only trust-critical, immutable states such as finalized matches and escrow operations, while the backend maintains all dynamic, high-volume, privacy-sensitive operational data.

Mobile Application

BidDrop's mobile application was built with React Native and Expo, supporting both Android and iOS users. WalletConnect integration allows secure user onboarding via blockchain wallets, replacing traditional credentials with cryptographically verified public addresses. The user interface supports a full cycle: request creation, matching, payment initiation, tracking, and confirmation of completed deliveries.

User Authentication and Wallet Integration

User authentication in BidDrop is performed through a secure wallet-based flow that relies on the user's blockchain wallet rather than traditional passwords. As illustrated in Figure [2], when initiating login, the mobile application requests permission from the user's chosen Substrate-compatible wallet: SubWallet[20] to access a public address. This public address becomes the user's unique and persistent identity within the system, eliminating the need for account creation, passwords, or any sensitive credential storage.

Once the user approves access to the wallet address, the application requests a one-time login challenge (nonce) from the backend. The wallet is then asked to sign this nonce, allowing the system to cryptographically verify that the user currently controls the private key associated with the selected public address—without ever exposing the private key itself. The signed message is returned to the backend, which validates the signature and issues a short-lived session token enabling access to the application. This authentication model provides a streamlined and highly secure login experience. It prevents password-related vulnerabilities, ensures that only the legitimate holder of the wallet can authenticate, and aligns with decentralized identity principles used in modern blockchain systems.

Dual-Stage Consensus: Aura and PoBA

BidDrop's blockchain employs a dual-stage consensus architecture in which Aura determines *who* authors each block and the PoBA mechanism determines *how* assignment proposals are evaluated, while the Aura-selected block author ultimately chooses the winning proposal for the slot.

At the start of every slot, all active nodes retrieve the current sets of open requests and available offers, execute the IDA* matching algorithm locally, and generate candidate assignment proposals. Each node submits its proposal on-chain through the `submit_proposal` extrinsic, ensuring that all bids for the slot are recorded before finalization begins.

When proposal submission for the slot is complete, the author selected by Aura for that slot executes the assignment selection step. Following the PoBA scoring rules, the block author inspects all recorded proposals, identifies the proposal with the highest total score, and selects it as the slot winner. Ties are resolved deterministically by choosing the earliest on-chain submission. If no valid proposals exist, the author finalizes the slot without an assignment. the block is still produced and contains routine metadata, maintaining liveness.

Before the block is sealed, the Substrate runtime enforces PoBA's protocol invariants. The runtime verifies that the chosen proposal is consistent, conflict-free, and correctly structured. Proposals with invalid identifiers, conflicting matches, or scoring inconsistencies are rejected. Only proposals that pass all runtime checks may be finalized and written into the block by the Aura-selected author.

The interaction between local IDA* matching, Aura's slot-based author selection, and PoBA's proposal finalization is illustrated in Figure [3]. As shown in the diagram, all nodes begin each slot with the same shared state of open requests and offers. Each node independently computes an IDA*-based proposal and submits it on-chain, after which Aura selects the block author responsible for invoking PoBA's finalization logic. The block author then evaluates all submitted proposals, selects the highest-scoring one, and then PoBA records the finalized assignment on-chain.

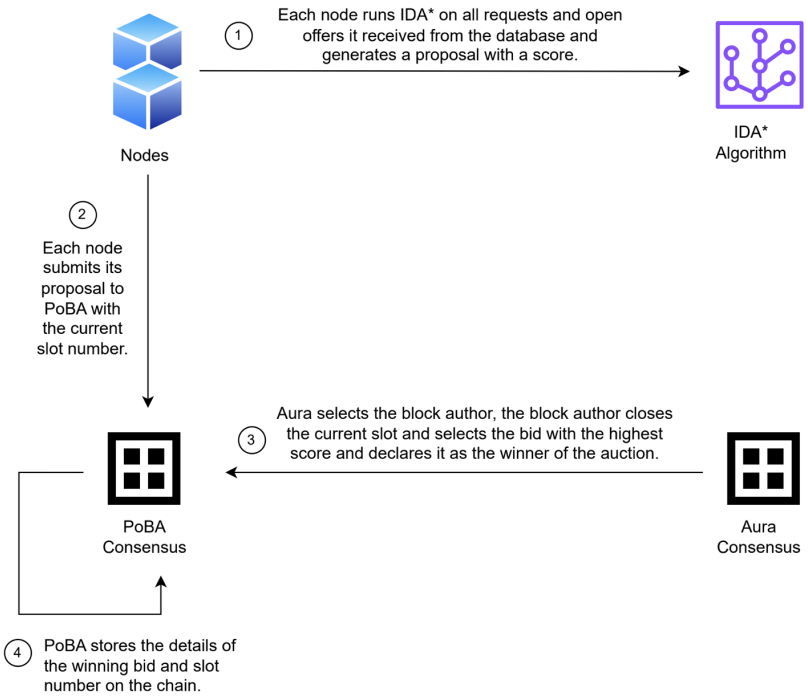

Figure 3: Interaction between per-node IDA* matching, Aura's block author selection, and PoBA's proposal finalization process. Each node independently generates a matching proposal, submits it on-chain, and the Aura-selected block author invokes PoBA to finalize the highest-scoring proposal for the slot.

Escrow Pallet Implementation

On-chain payment protection is managed by a dedicated Escrow pallet, implemented as an integral part of the BidDrop blockchain runtime rather than as an external smart contract. This native integration reduces external dependencies, avoids cross-chain complexity, and ensures that payment settlement inherits the same consensus and security guarantees as the matching process. When a request owner accepts an assignment, the corresponding payment amount is locked in escrow on-chain and cannot be accessed by either party. Upon delivery confirmation by the request owner, the funds are released to the driver. The pallet records all transaction details and emits domain-specific events, such as EscrowCreated and EscrowReleased, enabling reliable backend synchronization for both parties. In the same way, the Escrow pallet encodes the legal state transitions for payments, and the runtime ensures that only authorized actors can change payment status and that every status update follows the predefined lifecycle.

IDA* Matching Algorithm

The IDA* algorithm in BidDrop evaluates possible matches between requests and offers by considering three key factors: geographic distance, time-window compatibility, and price constraints. At each step, the algorithm searches for feasible pairings that minimize overall cost while maximizing the quality of the assignment. Its iterative deepening structure ensures an efficient exploration of the search space, producing an optimized proposal that reflects the best achievable combination of matches for the current slot.

6 End-to-End Flow Demonstration

For demonstration, this section presents a full end-to-end example of how the BidDrop platform operates in practice. The scenario follows a typical user journey within the system, beginning with secure wallet-based authentication and continuing through request creation, driver offer submission, on-chain matching via the PoBA consensus mechanism, and final payment settlement through the Escrow pallet. This example illustrates how the different components of the architecture interact—mobile application, backend services, database, and blockchain runtime—and how information flows between them during real operation. Screenshots from the BidDrop mobile application, backend database records, and the Substrate blockchain explorer are included to demonstrate how actions performed by users translate into stored data, on-chain events, and finalized state transitions. The scenario highlights the complete workflow supported by the platform and clarifies the roles of off-chain logic, on-chain consensus, and trustless payment enforcement in achieving a transparent and verifiable decentralized crowd-shipping system.

Step 1: Wallet-Based Log-In

The authentication process in BidDrop begins inside the mobile application, where the user selects the option to log in using a blockchain wallet. As shown in Figure [4], the login screen provides a "Connect with SubWallet" button that initiates the wallet-based authentication flow.

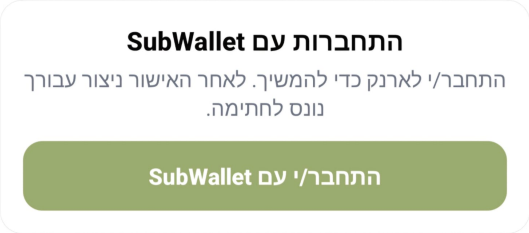

Figure 4: BidDrop login screen showing the "Connect with SubWallet" option.

After the user initiates the login flow, SubWallet opens via WalletConnect and displays the available wallet accounts on the device. As illustrated in Figure [5], the user selects the account they wish to authenticate with; its public address becomes the user's persistent identity within the platform.

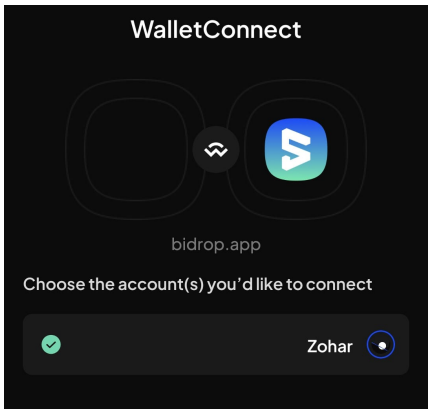

Figure 5: SubWallet WalletConnect interface showing account selection for authentication.

Once the account is selected, the wallet presents a signature request containing a one-time nonce issued by the backend. As seen in Figure [6], the user reviews the request and approves the signature, which provides cryptographic proof of wallet ownership without exposing the private key. This completes the secure, passwordless login flow.

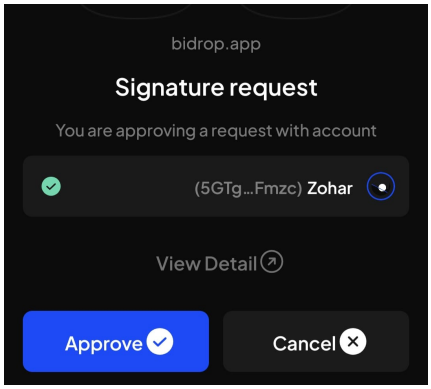

Figure 6: Signature request in SubWallet, showing the selected account and the nonce to be signed.

On the backend side, once the nonce signature is verified, the system either creates a new user record or retrieves an existing one based on the public wallet address. The user's public address and application-level details (such as their first name) are stored in the `users` table. As illustrated in Figure 7, this table links the blockchain identity provided by SubWallet to the internal user representation maintained by the BidDrop backend.

```
wallet_address  | first_name
----------------+------------
                |
5GTgng...2iFmzc | Zohar
```

Figure 7: Excerpt from the `users` table showing the stored wallet address and user name.

Step 2: Creating New Courier Availability Offer

After completing the authentication process, a Courier can publish a new availability offer, which represents the time and location in which the driver is able to provide delivery or ride-sharing services. As shown in Figure [8], the BidDrop application presents a structured form where the driver specifies the origin and destination, the available time window, the type of service they are willing to perform (parcel or ride), and the minimum price they expect to receive.

Figure 8: Screen for creating a new driver availability offer, including origin, destination, time window, service type, and minimum price.

Once the offer is submitted, it becomes an active availability entry within the driver's profile. As illustrated in Figure [9], the application displays the newly created availability card, confirming that the system has registered the offer with the specified route, time window, service type, and pricing preferences. This offer will later be considered by the PoBA-based matching process.

Figure 9: The driver's active availability offer as displayed in the application after submission.

On the backend side, the submitted availability offer is stored in the relational database and linked to the driver's user record. For the demonstration, we executed an SQL query that joins the `courier_offers` and `users` tables and retrieves the unique `offer_id` of the availability, the driver's name, a masked wallet address, and the origin and destination in English. As shown in Figure 10, the resulting row confirms that the active availability card displayed in the application corresponds to a concrete database record—including the specific `offer_id` and the Tel Aviv → Haifa route associated with the driver's on-chain identity.

offer_id	driver_first_name	driver_wallet	from_address_en	to_address_en
447d6d9b-0441-49e4-8080-036b24873fa4	Zohar	5GTgng...2iFmzc	Tel Aviv	Haifa

Figure 10: Result of an SQL query joining `users` and `courier_offers`, showing the driver's first name, masked wallet address, and origin/destination in English.

Step 3: Submitting a New Package Request and On-Chain Matching

In this step we demonstrate the complete flow of creating a new package request from the sender's side, waiting for the system to find a suitable driver, and observing how the matching process is executed and finalized on-chain by the PoBA consensus.

Figure [11] shows the in-app form used by the sender to publish a new delivery request, including pickup and drop-off addresses, a time window, and optional notes. After the request is submitted, it is recorded in the relational database. Since, for this demonstration, the sender and the driver are both logged in using the same account, the resulting database row naturally contains the same partially masked public wallet address. Figure [12] presents the corresponding row in the `requests` table, including the unique `request_id` (UUID), the sender's name, the shared wallet address, and the normalized origin and destination. This identifier will later allow us to verify that the request selected on-chain is indeed the same one created by the user.

Figure 11: New package request form in the BidDrop mobile application, where the sender specifies pickup and drop-off addresses, a time window, and optional notes before publishing the request.

request_id	requester_name	requester_wallet	from_address_en	to_address_en
fec850b0-3d9f-45a9-aa94-90cdab75ab41	Zohar	5GTgng...2iFmzc	Tel Aviv	Haifa

Figure 12: Database row for the submitted package request in the `requests` table, including the unique `request_id`, the requester name, a partially masked wallet address, and the normalized Tel Aviv–Haifa route.

Once the request is stored, the sender sees a "searching for a match" screen while the backend and the nodes process the current open requests and active driver availabilities. During this time, each node (Alice and Bob in our setup) independently constructs a PoBA proposal using the IDA* algorithm. Both proposals are submitted to the blockchain, resulting in two `poBA.ProposalSubmitted` events for the same slot, as shown in Figure 13. Figures [14] and [15] display the detailed events for Alice and Bob respectively, including the slot number, the total score of each proposal, and the number of matches produced.

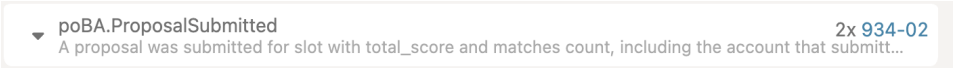

Figure 13: `poBA.ProposalSubmitted` events for the current slot, showing that two proposals were submitted—exactly one from each node (Alice and Bob).

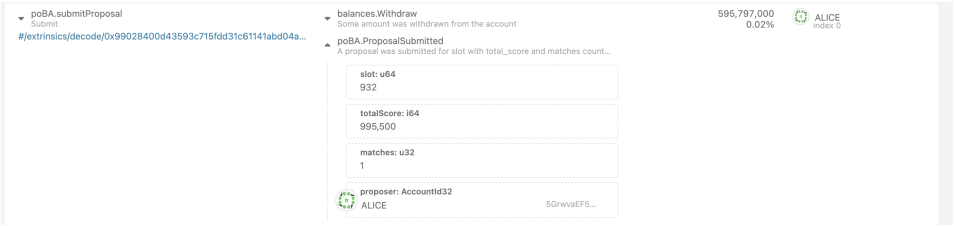

Figure 14: Decoded details of Alice's submitted proposal, including the slot number, the aggregated `totalScore`, the number of matches, and the proposer account (Alice).

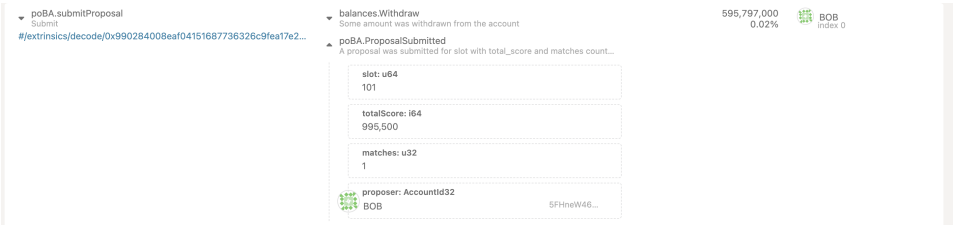

Figure 15: Decoded details of Bob's submitted proposal for a different slot, showing the same `totalScore` and single match, along with Bob's proposer account.

In this specific example, both nodes produced proposals with identical scores. Therefore, according to the PoBA decision rule, the first proposal included in the block wins the slot. As illustrated in Figure [16], Alice's proposal was the first to be submitted and is therefore selected as the winning proposal. The following block contains the `poBA.SlotFinalized` event (Figure [17]), indicating the finalized score and the number of matched pairs.

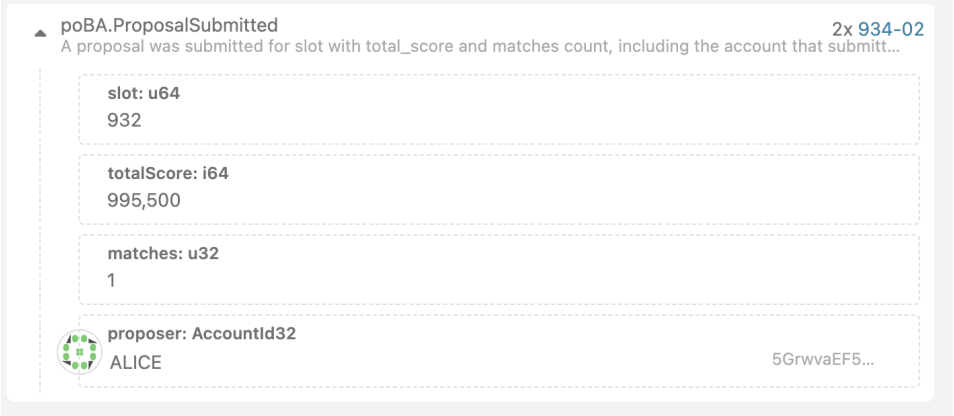

Figure 16: The `poBA.ProposalSubmitted` event for the winning proposal. In this example, Alice's proposal is selected as the winner because it was included first in the block and its score is equal to Bob's score.

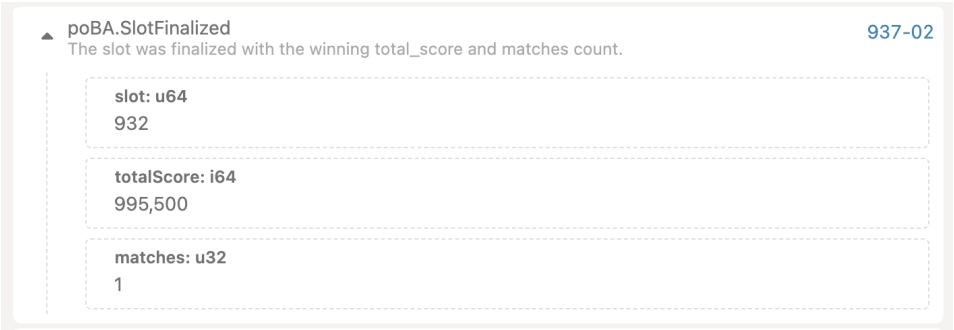

Figure 17: `poBA.SlotFinalized` event for the same slot, indicating that the slot was finalized with the winning `totalScore` and the number of matched pairs.

To further validate the correctness of the process, Figure [18] shows the on-chain contents of the finalized proposal. The block stores both the `requestUuid` and the `offerUuid` that were matched. These identifiers exactly match: (1) the availability identifier shown previously in Step 2 (Figure [10]), and (2) the request identifier stored in the database at the beginning of Step 3 (Figure [12]). This confirms that the blockchain finalized the same pair of entities created by the users in the application.

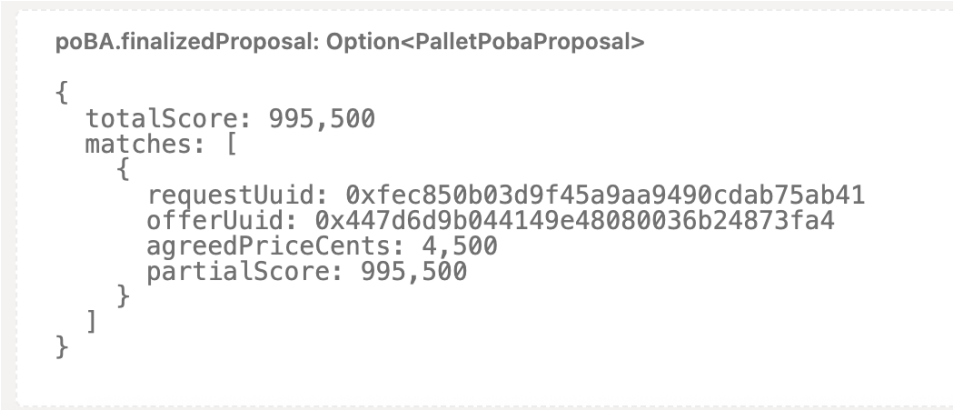

Figure 18: On-chain storage view of `poBA.finalizedProposal`, showing the finalized `totalScore` and the matched `requestUuid` and `offerUuid`. These identifiers correspond exactly to the availability shown in Step 2 and the request row shown at the beginning of Step 3.

Finally, once PoBA finalizes the slot, the application automatically updates the sender's interface to show that a suitable driver has been found. Figure [19] presents the success dialog, allowing the user to continue to the driver details screen.

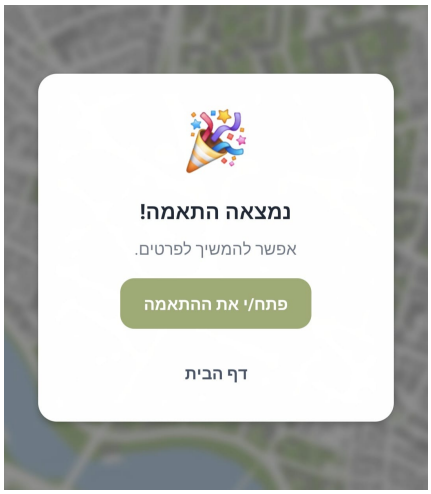

Figure 19: Client-side confirmation dialog in the BidDrop mobile application indicating that a suitable driver has been found for the sender's package request, allowing the user to continue to the detailed assignment view.

Step 4: Payment, Escrow Creation, Status Updates, and On-Chain Payment Release

Once a match is found and the sender views the assignment details, the application presents a dedicated screen summarizing the matched driver, the agreed route, and the total price. Figure [20] shows this in-app view, which also includes a "Proceed to Payment" button. When the sender taps this button, the backend submits an extrinsic to the blockchain that creates a new escrow instance for the matched request–offer pair.

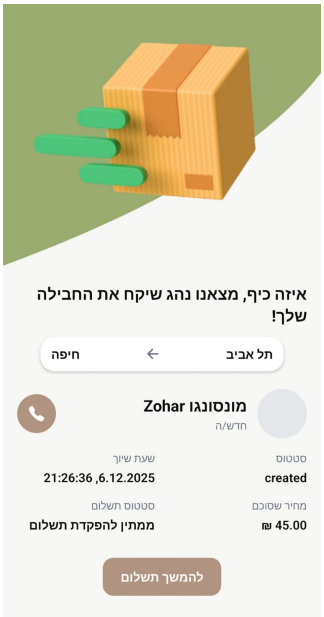

Figure 20: In-app assignment summary screen shown to the sender after a match is found, including the matched driver, route, price, and a button to initiate the payment process. Tapping this button triggers the creation of an on-chain escrow.

Immediately after this action, the chain emits an `escrow.EscrowCreated` event. Figure [21] displays the recorded on-chain escrow data, including the `escrowId`, the `requestUuid` and `offerUuid` which match precisely the identifiers presented in the earlier steps, and the involved accounts: the payer (the package sender) and the receiver (the matched driver). The agreed amount (in cents) is also stored on-chain as part of the escrow record.

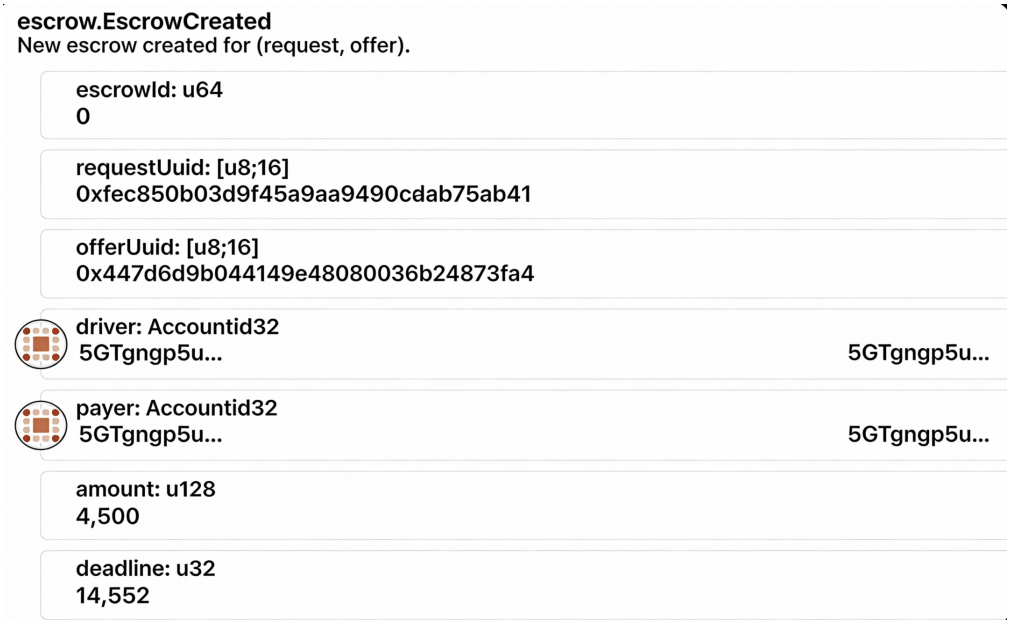

Figure 21: The blockchain event `escrow.EscrowCreated`, emitted immediately after the sender confirms payment. The event includes the `escrowId`, the matched `requestUuid` and `offerUuid` (identical to those shown in previous steps), the payer and receiver accounts, and the agreed amount locked in the escrow.

Following the creation of the escrow, the application presents an informational screen explaining the payment process and clarifying that no credit-card charge has been made. Instead, a logical escrow is locked on the blockchain until delivery is completed. Figure [22] illustrates this explanation screen shown to the sender.

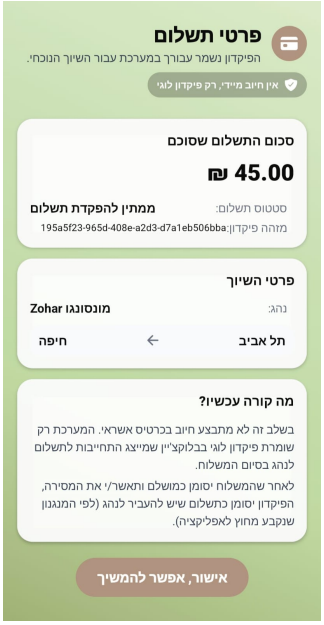

Figure 22: Information screen explaining the escrow-based payment flow. No credit card charge is made at this point; instead, a logical escrow is created on-chain and will be released only after the delivery is completed and confirmed by the sender.

Once the delivery is performed, the driver updates the shipment status to "completed." Figure [23] shows the driver-side interface confirming this update. Subsequently, the sender is prompted to confirm receipt of the package. Figure [24] shows the corresponding confirmation screen from the sender's perspective.

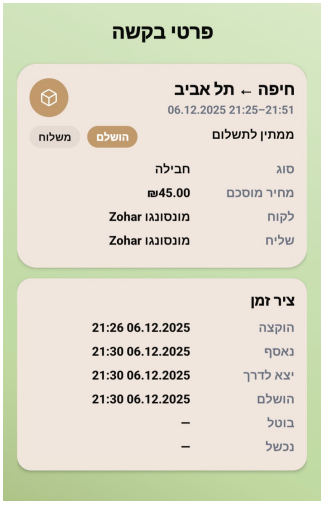

Figure 23: Driver-side interface showing the status update to "completed" once the package has been delivered. This update is required before the sender can confirm receipt.

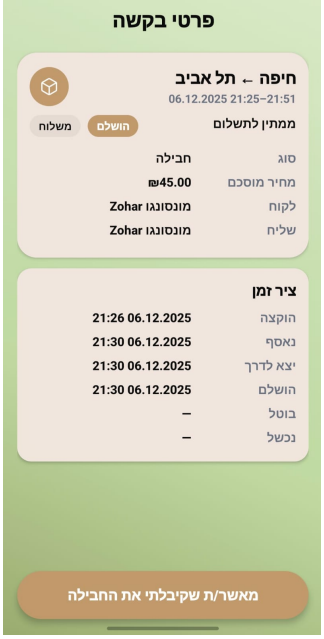

Figure 24: Sender confirmation screen indicating that the package has been received. Once this confirmation is submitted, the backend triggers the on-chain payment release procedure.

When the sender confirms that the package has been received, the on-chain release process is triggered. The blockchain then emits an `escrow.PaymentReleased` event, finalizing the escrow and transferring the locked amount to the driver. As shown in Figure [25], the released event contains the same `escrowId` created earlier along with the corresponding payment amount, demonstrating the full end-to-end escrow lifecycle from creation to release.

Figure 25: The blockchain event `escrow.PaymentReleased`, emitted after the sender confirms receipt. The event includes the same `escrowId` that was created earlier and the released payment amount, demonstrating the full escrow lifecycle from creation to release.

7 Discussion and Conclusion

The results of this work show that embedding PoBA directly into the blockchain consensus layer fundamentally changes how matching can be performed in decentralized crowd-

shipping systems. Instead of relying on a central coordinator, assignment decisions become part of a transparent and auditable consensus process, with nodes collectively determining optimal matches and recording them immutably on-chain. Together with a native runtime Escrow pallet, the system demonstrates that both matching and payment settlement can be handled trustlessly within the protocol itself, creating an end-to-end flow from request creation to delivery confirmation without the need for a central authority.

The BidDrop platform demonstrates the practical feasibility of this architecture: the system functions as an integrated pipeline connecting the mobile application, backend services, PoBA-based matching, and on-chain escrow enforcement. This design provides strong transparency and auditability guarantees, yet it also introduces trade-offs, including higher technical complexity, the requirement to operate blockchain nodes, and usability considerations associated with wallet-based authentication.

Although the system introduces an innovative consensus-based approach to matching and payment enforcement, it does not fully resolve all trust, scalability, or security challenges identified, but rather demonstrates a viable direction that future work must expand upon. The design therefore prioritizes verifiability and decentralization over operational simplicity, making it suitable for environments where fairness, trust, and transparency are essential.

While successful as a proof-of-concept, the system has several limitations. The evaluation was conducted at small scale, without formal adversarial or economic analysis, and without large-scale stress testing. The current prototype assumes honest participants, controlled conditions, and relatively simple request structures. These constraints mean that the results, while promising, are not yet indicative of performance or robustness in large, real-world deployments. Additionally, although the system adopts a decentralized blockchain-based architecture, it still relies on an external backend server for operational data management and synchronization. This hybrid design means the platform is not fully decentralized, as certain components—such as request storage, offer lifecycle management, and event handling—remain off-chain. This dependency introduces a partial trust assumption and highlights an important limitation of the current design.

Future work should include scalability benchmarking of PoBA and the IDA* algorithm, formal security modeling, incentive analysis, and large-scale pilot deployments involving hundreds of users. Additional directions include exploring privacy-preserving data-management designs, extending the matching logic to more complex routing scenarios, and integrating the platform with external logistics and navigation services. Another important direction is the incorporation of driver rating and reputation mechanisms into the matching process, allowing IDA* to factor user trust and historical performance into allocation decisions. Such mechanisms could significantly enhance reliability and user confidence by ensuring that high-quality couriers are prioritized in the assignment process. A further direction for future development is reducing reliance on the external backend service by progressively migrating more operational state and validation logic on-chain or to decentralized storage layers. Achieving a higher degree of decentralization would remove remaining trust assumptions and bring the system closer to a fully trustless coordination model. Together, these enhancements would support the evolution of BidDrop toward a mature, deployable decentralized logistics solution.

9 References

[1] H. Song, N. Zhu, R. Xue, J. He, K. Zhang, and J. Wang, "Proof-of-Contribution consensus mechanism for blockchain and its application in intellectual property protection," *Information Processing Management*, vol. 58, no. 3, 2021. `https://doi.org/10.1016/j.ipm.2021.102507`

[2] P. Bajec and D. Tuljak-Suban, "Exploring barriers and enablers of last-mile crowdshipping: understanding the motivation of potential groups of crowdshippers in the Slovenian context," *European Transport Research Review*, vol. 16, no. 70, 2024. DOI: 10.1186/s12544-024-00693-7.

[3] T. V. Le, A. Stathopoulos, T. Van Woensel, and S. V. Ukkusuri, "Supply, demand, operations, and management of crowd-shipping services: A review and empirical evidence," *Transportation Research Part C: Emerging Technologies*, vol. 103, pp. 83–103, 2019. DOI: 10.1016/j.trc.2019.03.023.

[4] A. Sreekumar, A. Sakthivelu, and L. Kiesling, "Auction Theory and Device Bidding Functions for Transactive Energy Systems: A Review," *Current Sustainable/Renewable Energy Reports*, vol. 10, pp. 102–111, 2023. DOI: 10.1007/s40518-023-00217-2.

[5] F. Yang, W. Zhou, Q. Wu, R. Long, N. N. Xiong, and M. Zhou, "Delegated Proof of Stake With Downgrade: A Secure and Efficient Blockchain Consensus Algorithm With Downgrade Mechanism," *IEEE Access*, vol. 7, pp. 118541–118555, 2019. DOI: 10.1109/ACCESS.2019.2935149.

[6] T. Nododile and C. N. Nyirenda, "Advancing Blockchain-Enabled InterPlanetary File System with Substrate for Distributed Data Storage," *2024 International Conference on Emerging Trends in Networks and Computer Communications (ETNCC)*, Windhoek, Namibia, 2024, pp. 1–8. DOI: 10.1109/ETNCC63262.2024.10767518.

[7] X. Zhang, R. Li, Q. Wang, and S. Duan, "Time-manipulation Attack: Breaking Fairness against Proof of Authority Aura," *Proceedings of the ACM Web Conference 2023 (WWW '23)*, Austin, TX, USA, April 30–May 04, 2023, pp. 2076–2086. DOI: 10.1145/3543507.3583252.

[8] Z. Hussein, M. A. Salama, and S. A. El-Rahman, "Evolution of blockchain consensus algorithms: a review on the latest milestones of blockchain consensus algorithms," *Cybersecurity*, vol. 6, no. 30, 2023. DOI: 10.1186/s42400-023-00163-y.

[9] E. Finkelstein, "Smart" Contract Law – Between Technological Advancement and Regulation," *Ma'asei Mishpat*, vol. 12, pp. 349–372, 2021. `https://en-law.tau.ac.il/sites/law-english.tau.ac.il/files/media_server/law_heb/Maasei_Mishpat/12/11%20elad%20finkelstein.pdf`

[10] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-Based Decentralized Auctions for Robust Task Allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009. DOI: 10.1109/tro.2009.2022423. `http://hdl.handle.net/1721.1/52330`

[11] A. Rechavi and E. Toch, "Crowd logistics: Understanding auction-based pricing and couriers' strategies in crowdsourcing package delivery," *Journal of Intelligent Transportation Systems*, vol. 24, no. 2, pp. 129–144, 2020. DOI: 10.1080/15472450.2020.1797503.

[12] T. Akamatsu and Y. Oyama, "A market-based efficient matching mechanism for crowdsourced delivery systems with demand/supply elasticities," *arXiv preprint*, arXiv:2412.20395, 2024. https://arxiv.org/pdf/2412.20395.pdf

[13] R. J. Tapia, N. L. Significance, and Others, "A disaggregate model of passenger-freight matching in crowdshipping services," *Transportation Research Part A: Policy and Practice*, vol. 169, 2023. https://significance.nl/wp-content/uploads/2023/07/2023-RTA-A-disaggregate-model-of-passenger-freight-matching-in-crowdshipping-services.pdf

[14] P. Chen and S. M. Chankov, "Crowdsourced delivery for last-mile distribution: An agent-based modelling and simulation approach," *2017 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, pp. 1271–1275, 2017. Available: https://doi.org/10.1109/IEEM.2017.8290097

[15] A. Punel and A. Stathopoulos, "Crowd-shipping delivery performance from bidding to delivering," *Research in Transportation Business & Management*, vol. 24, pp. 68–77, 2017. Available: https://www.sciencedirect.com/science/article/abs/pii/S221053952030116X

[16] X. Su, X. D´efago, M. Larangeira, K. Mori, T. Oda, Y. Tamura, and K. Tanaka, "Double Auction Meets Blockchain: Consensus from Scored Bid-Assignment," *Double Auction Meets Blockchain*, vol. 135, pp. 333–347, 2022. Available: https://eprint.iacr.org/2022/1471.pdf

[17] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-Based Decentralized Auctions for Robust Task Allocation," *MIT DSpace Technical Report*, 2009. Available: https://dspace.mit.edu/bitstream/handle/1721.1/52330/Choi_Consensus-Based-Decentralized.pdf

[18] W. Jiang, "Analysis of Iterative Deepening A* Algorithm," *IOP Conference Series: Earth and Environmental Science*, vol. 693, Article 012028, 2021. Available: https://doi.org/10.1145/3543507.3583252

[19] Parity Technologies, "Substrate Blockchain Framework." https://substrate.io/.

[20] SubWallet. "SubWallet Documentation." Available at: https://subwallet.app/